arXiv:2410.05265v2 [cs.LG] 27 Jan 2025

PrefixQuant: Eliminating Outliers by Prefixed Tokens for Large Language Models Quantization

Mengzhao Chen [1] **Yi Liu** **Jiahao Wang** [1] **Yi Bin** [2] **Wenqi Shao** [3] **Ping Luo** [1]

Abstract

Existing weight-activation quantization methods for Large Language Models (LLMs) primarily address channel-wise outliers but often neglect token-wise outliers, which limits the accuracy of quantized models. In this work, we propose PrefixQuant, a novel quantization method that achieves state-of-the-art performance across various precision levels (W4A4KV4 and W4A8KV4) and granularities (dynamic and static quantization) by effectively isolating token-wise outliers. First, PrefixQuant eliminates token-wise outliers by prefixing outlier tokens in the KV cache, a process that is training-free and highly efficient (*e.g.*, 1 minutes for Llama-3-70B). Second, PrefixQuant introduces new trainable parameters for block-wise training to compensate for quantization error. Our experiments show that PrefixQuant significantly outperforms existing dynamic quantization methods, even under coarser static quantization settings. For instance, PrefixQuant achieves an average accuracy improvement of $+3.08$ and $+2.85$ points over SpinQuant (dynamic quantization) on five zero-shot reasoning tasks under dynamic and static quantization settings, respectively, on W4A4KV4 Llama-3-8B. Additionally, we demonstrate up to $2.74\times$ prefilling speedup and $2.16\times$ decoding speedup for LLMs using W4A4 PrefixQuant. Our code is available at `https://github.com/ChenMnZ/PrefixQuant`.

1. Introduction

Recently, Large Language Models (LLMs)(Touvron et al., 2023; Bubeck et al., 2023) demonstrate remarkable capabilities across various tasks. However, their large parameters and computational demands pose significant challenges for deployment. This makes quantization (Frantar et al., 2022; Lin et al., 2023; Shao et al., 2023) a crucial technology for reducing memory usage and speeding up inference (Yuan et al., 2024).

[1]The University of Hong Kong [2]Tongji University [3]Shanghai AI Laboratory. Correspondence to: Wenqi Shao <shaowenqi@pjlab.org.cn>, Ping Luo <pluo@cs.hku.hk>.

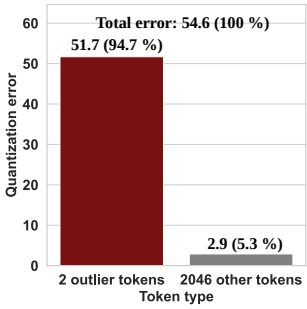
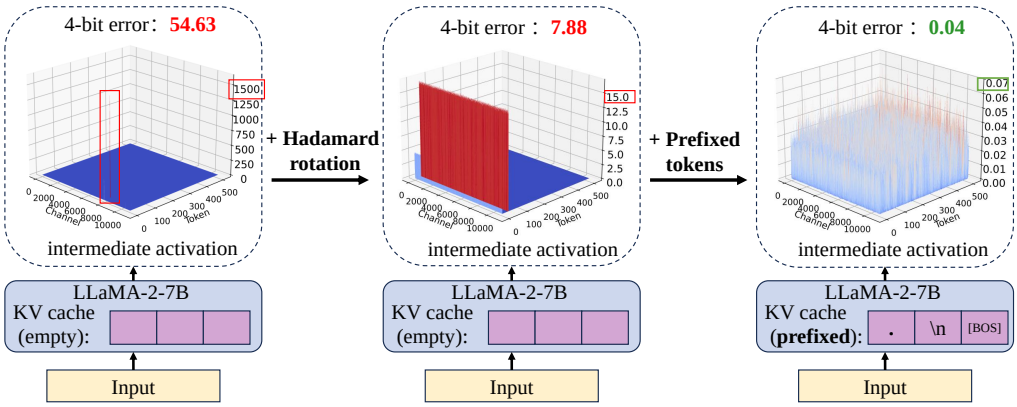


Figure 1: **4-bit per-token dynamic quantization error in 2048 input context length.** Two outlier tokens account for 94.7% of quantization error , while the remaining 2046 tokens contribute only 5.4%. Quantization error is measured in the output of Llama-2-7B 2-nd transformer block through mean square error (MSE).

Despite advancements, outliers in LLMs activations can lead to significant quantization errors and accuracy degeneration. Many current methods address this by focusing on alleviating channel-wise outliers (Dettmers et al., 2022) through techniques like channel-wise scaling (Xiao et al., 2023a; Shao et al., 2023; Wei et al., 2023a), mixed-precision quantization (Dettmers et al., 2022; Zhao et al., 2023), Hadamard rotation (Ashkboos et al., 2024b; Liu et al., 2024b), or channel-level assembly (Liu et al., 2023). However, activations of LLMs include not only channel-wise outlier but also some massive activation (Sun et al., 2024) only occur in a few tokens, and can be termed as token-wise outliers. For example, Figure 1 shows that 2 outlier tokens among 2048 tokens contribute 94.7% of the quantization error. Figure 2(a) provides a more detailed analysis, revealing that a few tokens have extreme values exceeding 1,000, resulting in a quantization error of 54.63. The current state-of-the-art method, Hadamard rotation(Ashkboos et al., 2024b), redistributes outlier values across all channels, reducing the maximum value of outlier tokens from over 1,000 to approximately 15 (see Figure 2 (b)). However, the magnitude of outlier tokens remains hundreds of times larger than normal



(a) Original (b) Hadamard Rotation (c) PrefixQuant (ours)

Figure 2: **Comparison of proposed PrefixQuant with existing methods.** This figure shows the intermediate input activation of the 2-nd down_proj linear layer in Llama-2-7B using different methods. Quantization error is measured in the output of Llama-2-7B 2-nd transformer block through mean square error with 4-bit per-token dynamic quantization. The original distribution has significant outliers larger than 1,500 (left), leading 54.63 quantization error. The previous method with Hadamard rotation (Ashkboos et al., 2024b) reduces outliers to nearly 15 (middle) but still suffers from 7.88 quantization error. We propose PrefixQuant (right), which prefixes some specific tokens in KV cache to isolate outliers, reducing the maximum to nearly 0.07, significantly improving quantization error to 0.04.

tokens, leading to a quantization error of 7.88.

In this paper, we propose PrefixQuant, an efficient method to isolate token-wise outliers for more accurate quantization. PrefixQuant is based on a key observation: **Prefixing high-frequency outlier tokens at the beginning of the input sequence constrains token-wise outliers to only occur in the prefixed tokens.** Since prefixed tokens remain consistent across all inputs, PrefixQuant performs offline prefilling of these tokens and stores their KV cache. This stored KV cache can then be reused for all inputs, effectively avoiding token-wise outliers during the forward pass. Furthermore, the detection of prefixed tokens is efficient and does not require any retraining, unlike prior methods (Sun et al., 2024; Bondarenko et al., 2024). For example, this process completes in just 12 seconds for Llama-2-7B. As shown in Figure 2(c), PrefixQuant effectively eliminates outliers and reduces the quantization error from 7.88 (using QuaRot) to 0.04. Additionally, we introduce a block-wise fine-tuning (Shao et al., 2023; Chen et al., 2024a) to compensate for quantization error by jointly training weights and quantization parameters. For static activation quantization, the quantization parameters are inherently trainable. However, dynamic activation quantization lacks trainable quantization parameters. To address this, we propose learnable activation clipping to enable training for dynamic activation quantization.

Since PrefixQuant is compatible with various quantization schemes, we introduce two settings for PrefixQuant (see Table 1): O1 for dynamic quantization and O2 for static quantization. Experiments show that PrefixQuant significantly outperforms existing methods (Ashkboos et al., 2024b; Xiao et al., 2023a; Lin et al., 2024b) under the same dynamic quantization setting. Furthermore, PrefixQuant even surpasses prior dynamic quantization methods while utilizing the more efficient static quantization. For instance, PrefixQuant achieves an average accuracy improvement of $+3.08$ and $+2.85$ points over SpinQuant (dynamic quantization) on five zero-shot reasoning tasks under dynamic and static quantization settings, respectively, on W4A4KV4 Llama-3-8B. To the best of our knowledge, PrefixQuant is the first method to surpass prior per-token dynamic quantization methods (Ashkboos et al., 2024b; Xiao et al., 2023a; Lin et al., 2024b) using the coarser per-tensor static quantization. We also benchmark the end-to-end inference of W4A4 quantization, where PrefixQuant achieves a $2.74\times$ prefilling speedup and a $2.16\times$ decoding speedup compared to FP16 models. We hope PrefixQuant inspires future developments in LLM compression.

2. Related Works

This section discusses works related to enhancing quantization accuracy by eliminating activation outliers.

Channel-Wise Outliers. Activation outliers often recur in the same channels across tokens. (Dettmers et al., 2022) addresses this by isolating outlier channels with 16-bit precision, while Atom (Zhao et al., 2023) and QUIK (Ashkboos et al., 2023) adopt similar mixed-precision strategies. Other methods, like SmoothQuant (Xiao et al., 2023a), OmniQuant (Shao et al., 2023), and Outlier Suppression (Wei et al., 2022; 2023b), scale activations to weights on a

Table 1: Quantization setting of the baselines and PrefixQuant. All group-wise quantization set group size as 128. For PrefixQuant, O1 is the same as existing methods for fair comparisons, and O2 is more efficient than O1 (*i.e.* lower latency).

Method	Weight	Activation	KV Cache
SmoothQuant	per-channel	per-token dynamic	per-token dynamic
Atom	group-wise	group-wise dynamic	group-wise dynamic
QoQ;QuaRot;SpinQuant;SpinQuant	per-channel	per-token dynamic	group-wise dynamic
PrefixQuant-O1	per-channel	per-token dynamic	group-wise dynamic
PrefixQuant-O2	per-channel	per-tensor static	per-head static

channel-wise basis. QLLM (Liu et al., 2023) splits outlier channels into sub-channels, and QuaRot (Ashkboos et al., 2024b) redistributes outliers using random Hadamard rotation, later improved by SpinQuant (Liu et al., 2024b), which trains the orthogonal matrix. In contrast, our work focuses on token-wise outliers and is orthogonal to these channel-wise methods.

Token-Wise Outliers. The SoftMax function in self-attention prevents zero attention scores, causing unnecessary scores for special tokens and leading to token-wise outliers (Sun et al., 2024; Xiao et al., 2023b; Gu et al., 2024). StreamingLLM (Xiao et al., 2023b) and LM-infinite (Han et al., 2023) retain initial tokens for long-context generation, while our PrefixQuant isolates outliers by carefully selecting prefixed tokens in the KV-cache for quantization. Unlike training-based methods (Bondarenko et al., 2024; Sun et al., 2024) that modify SoftMax behavior or add attention bias, our PrefixQuant isolates outliers without retraining. Closest works, like QFeP (Yang et al., 2024) and CushionCache (Son et al., 2024), rely on costly grid searches (e.g., 12 hours for Llama-3-8B), while PrefixQuant completes this in 12 seconds. Furthermore, unlike prior works (Sun et al., 2024; Son et al., 2024) focusing on large-value outliers, PrefixQuant also identifies extremely small-value outliers in self-attention queries and keys.

3. Preliminaries

Quantization in LLMs involves weight, activation, and KV cache quantization. Weight quantization (Chen et al., 2024a) and KV cache quantization (Liu et al., 2024a) reduce memory usage and speed up memory-bound computations (Yuan et al., 2024). Combining weight and activation quantization enables low-bit matrix manipulation to accelerate computation-bound tasks (Yuan et al., 2024). Specifically, the quantization process is:

$$\mathbf{X}_{\texttt{INT}} = \text{clamp}\left(\lfloor\frac{\mathbf{X}}{s}\rceil + z, 0, 2^N - 1\right), \tag{1}$$

$$\text{where} \quad s = \frac{\gamma\text{max}(\mathbf{X}) - \beta\text{min}(\mathbf{X})}{2^N - 1}, z = -\lfloor\frac{\beta\mathbf{min}(\mathbf{X})}{s}\rfloor \tag{2}$$

where $\lfloor\cdot\rceil$ denotes rounding operation, N is the target bit number, $\mathbf{X}_{\texttt{INT}}$ and $\mathbf{X}$ are the quantized integer and full-precision tensor, respectively. $\mathbf{s}$ and $\mathbf{z}$ are quantization parameters, for the step size and zero values, respectively. $\gamma \in [0, 1]$ and $\beta \in [0, 1]$ are clipping factors.

Dynamic and Static. Activation quantization is divided into dynamic and static quantization based on how quantization parameters are calculated. Specifically, dynamic quantization calculates s and z online during inference, offering better adaptability to different distributions. In contrast, static quantization precomputes s and z offline through calibration datasets, leading to more efficient inference and more feasible operator fusion (Nagel et al., 2021).

Initialization of Quantization Parameters. The classical approach uses max–min initialization, where both γ and β are set to 1. To better balance clipping error and rounding error (Lin et al., 2023; 2024b), we initialize γ and β using MSE-based grid search for both weight and activation quantization in our experiments. Specifically, for per-token dynamic quantization, γ and β are shared across all tokens within the same layer. For per-tensor static quantization, we directly perform a grid search for the quantization parameters s and z instead of optimizing the clipping factors.

Hadamard Rotation. Random Hadamard rotation (Ashkboos et al., 2024b; Liu et al., 2024b) addresses channel-wise outliers. Our method focus on removing token-wise outliers. Therefore, We build our method upon the Hadamard rotation technique, and the detailed is provided in Sec. B.

4. PrefixQuant

In this section, we present the proposed PrefixQuant methods. Sec.4.1 describes the characteristics of outlier tokens in LLMs. Sec.4.2 proposes a solution to isolate these outlier tokens, creating a more quantization-friendly distribution. Finally, Sec.4.3 introduces block-wise optimization to further reduce quantization error.

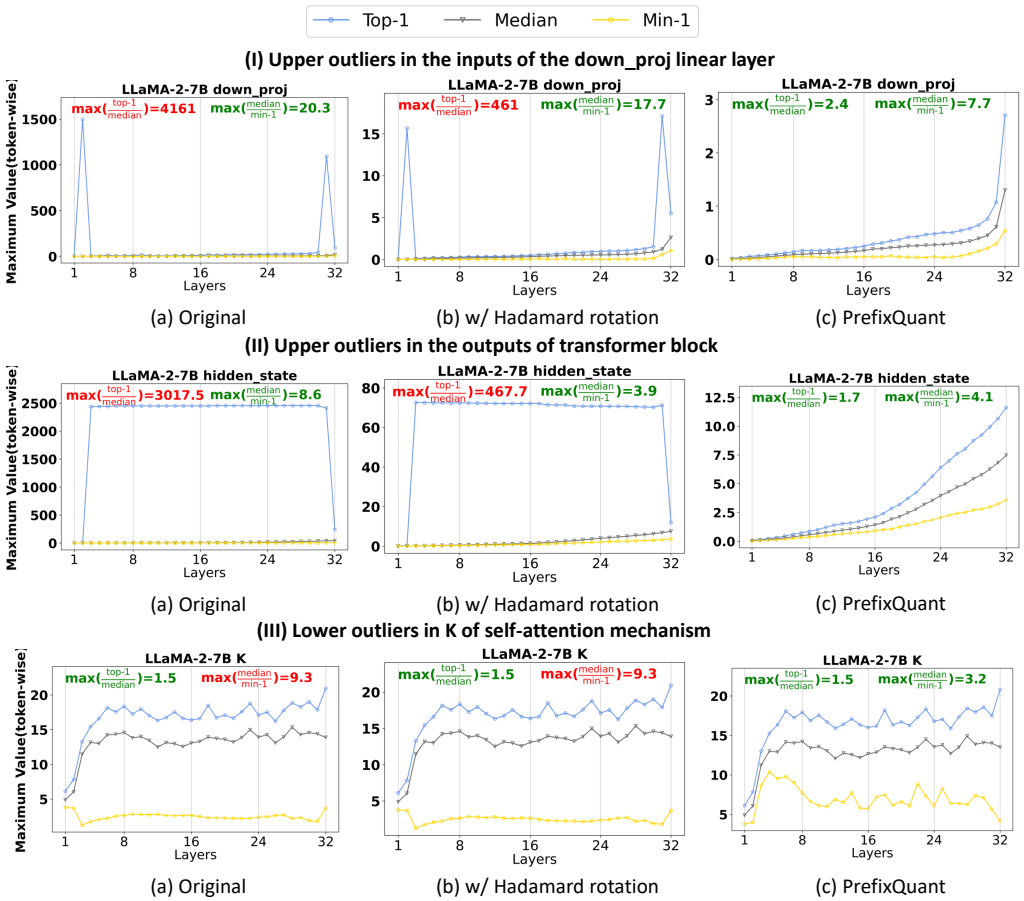


Figure 3: Example of token-wise outliers. We present (I)(II) upper outliers and (III) lower outliers. Top-1, Medium, Min-1 indicate the largest, median, and smallest values among token-wise maximum values, respectively. We also calculate the the ratios of $\frac{\text{Top-1}}{\text{Median}}$ and $\frac{\text{Median}}{\text{Min-1}}$ in each layer, and report the maximum ratio across all layers . A lower ratio indicates a more uniform distribution. we take Llama-2-7B as an example here, more visualizatiosn about other models can be find in Sec.H.

4.1. Deep Exploration of Outlier Tokens

Both channel-wise and token-wise outliers can cause significant quantization error. While channel-wise outliers have been thoroughly explored and addressed in prior research (Ashkboos et al., 2024b), this discussion focuses on token-wise outliers.

Definition of Outlier Token. Let $\mathbf{X} \in \mathbb{R}^{T \times C}$ represent the absolute values of a token sequence, where T is the number of tokens and C is the dimension size. We compute the token-wise maximum values $\mathbf{M} \in \mathbb{R}^{T}$, where each element $\mathbf{M}_i$ indicates the maximum value of the i-th token. The outlier degree of a token is then measured by comparing its maximum value to the median of $\mathbf{M}$:

$$R_i = \frac{\mathbf{M}_i}{\text{median}(\mathbf{M})}. \tag{3}$$

Then, the i-th token is classified as an outlier if R_i deviates significantly from 1 (*i.e.*, either much larger or much smaller than 1). Specifically, we define an *upper outlier token* when $R_i > \eta_1$ and a *lower outlier token* when $R_i^{-1} > \eta_2$. In our experiments, we set $\eta_1 = 64$ and $\eta_2 = 8$.

Visualization of Outlier Tokens. To better illustrate the outlier degree, we further define $\max\left(\frac{\text{top-1}}{\text{median}}\right)$ and $\max\left(\frac{\text{median}}{\text{min-1}}\right)$ as the maximum R_i and maximum R_i^{-1} across different layers, respectively. Following the definition of R_i in Eq. (3), a larger $\max\left(\frac{\text{top-1}}{\text{median}}\right)$ indicates the presence of extreme upper outliers, while a larger $\max\left(\frac{\text{median}}{\text{min-1}}\right)$ reflects the presence of extreme lower outliers. Specifically, we identify the following outlier tokens:

1) Upper outlier tokens in inputs of down_proj layers and outputs of transformer blocks. As shown in Figure 3(I.a), the input activations of the down_proj layers exhibit signifi-

cant upper outliers, with $\max\left(\frac{\text{top-1}}{\text{median}}\right) = 4161$. Although Hadamard rotation (Figure 3(I.b)) reduces this ratio to 461, it still indicates a large gap compared to normal tokens. A similar phenomenon is observed in the outputs of transformer blocks, as shown in Figure 3(II). These outliers not only lead to larger quantization errors but also cause instability during block-wise fine-tuning.

2) Lower outlier tokens in Q/K. In Figure 3(II.a), $\mathbf{K}$ displays a distinct outlier pattern compared to the inputs of linear layers. Instead of having large magnitudes, some tokens exhibit extremely small values. Specifically, $\mathbf{K}$ has $\max\left(\frac{\text{top-1}}{\text{median}}\right) \approx 1.5$, but $\max\left(\frac{\text{median}}{\text{min-1}}\right) > 9$. Furthermore, as shown in Figure 3(II.b), Hadamard rotation does not mitigate these lower outliers. Similar lower outliers are also observed in $\mathbf{Q}$, as shown in Figure 10.

Additionally, we observe that both upper outlier tokens and lower outlier tokens correspond to tokens at the same position in the sequence, but they exhibit different patterns in different modules. Therefore, we focus on analyzing upper outlier tokens due to their stronger prominence and ease of detection.

Characters of outlier tokens. We further investigate the characteristics of these outlier tokens, including the number of outlier tokens in an input sequence, their positions, and their content (text):

- **Number**: We determine the number of outlier tokens in a small calibration dataset. Specifically, we compute the average outlier token count $\mathbf{O} \in \mathbb{R}^b$ for each transformer block according to compare Eq (3) with the outlier threshold η_1, where b is the total number of transformer blocks. Since outlier tokens are nearly consistent across layers that contain them, we simply set the number of outlier tokens as $o = \lceil \max(\mathbf{O}) \rceil$. Consistent with Massive Attention (Sun et al., 2024), we find that outlier tokens appear in only a small fraction of positions (*e.g.* 2 for Llama-2-7B) within the input sequence, as shown in Figure 4a.
- **Position**: We observe that the initial tokens are outlier tokens across almost all models, aligning with findings on attention sinks (Xiao et al., 2023b). Additionally, Figure 4c shows that, apart from the initial tokens, some other tokens near the beginning of the sequence are also outlier tokens. Unlike outlier channels, which occur at fixed channel indices (Dettmers et al., 2022), the positions of outlier tokens depend on the input sequence and vary significantly. As a result, it is not feasible to identify outlier tokens offline for mixed-precision quantization.
- **Content (text)**: Initial tokens are consistently outlier tokens, regardless of their content. Thus, we focus on outlier tokens that are not initial tokens to analyze their content. Some models, such as Llama-3-8B and Qwen-2-7B, exhibit outlier tokens only at the initial positions. However, certain models display outlier tokens not only at the start of the input sequence but also in low-semantic tokens. For example, Llama-2-7B shows outlier tokens in both initial and delimiter tokens (e.g., ." or \n"), as illustrated in Figure 4b. Notably, tokens corresponding to the same text may exhibit different patterns depending on their position in the sequence. For instance, low-semantic tokens may behave as outlier tokens at the front of the sequence but appear as normal tokens in other positions.

4.2. Prefixed Outliers

Given that the number of outlier tokens is limited and they typically occur at the beginning of the input sequence, we propose a method to prefix high-frequency outlier tokens in the input sequence. This approach constrains outlier tokens to the prefixed tokens. Furthermore, these prefixed tokens can be directly stored in the KV cache, enabling more efficient computation, as shown in Figure 2.

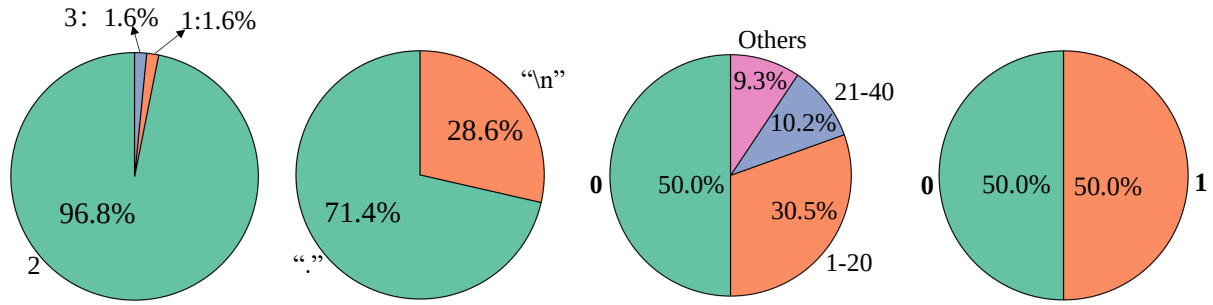
What token is added as a prefix. To determine which tokens to add as a prefix, we firstly analyze the number of outlier tokens o. We find that prefixing the top-o high-frequency[1] outlier tokens successfully constrains outliers to the prefixed tokens, as illustrated in Figure 4d. For special cases, such as models like Llama-3-8B and Qwen-2-7B where outlier tokens only appear as initial tokens, we set the prefix token to "[BOS]". Additionally, for consistency, we also include "[BOS]" as the last prefixed token for all models. The detailed prefixed tokens used for different models are listed in Table 5.

Computation of prefixed tokens in KV cache. In the auto-regressive inference pipeline of LLMs, we directly store these prefixed tokens in the KV cache to prevent new outlier tokens from being generated during inference. Specifically, given the input query, key, and value matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times C}$, the self-attention mechanism with prefixed tokens in the KV cache is formulated as:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}; \mathbf{k}', \mathbf{v}') = \text{Softmax}\left(\frac{\mathbf{Q}\left[\mathbf{K}^T \ \mathbf{k}'\right]}{\sqrt{d}}\right)\begin{bmatrix}\mathbf{V} \\ \mathbf{v}'^T\end{bmatrix} \tag{4}$$

Here, $\mathbf{k}', \mathbf{v}' \in \mathbb{R}^{o \times C}$ are the prefixed tokens stored in the KV cache. We compute $\mathbf{k}'$ and $\mathbf{v}'$ during a one-time prefilling process using the full-precision model. These prefixed tokens are then stored in the KV cache and reused during inference by quantized models. Notably, the prefixed tokens in the KV cache remain in full precision, even when used with quantized models.

[1]The frequencies are calculated excluding the initial token.



(a) Number of outlier tokens (b) Content of outlier tokens (exclude position 0) (c) Position index of outlier tokens (d) Position index of outlier tokens **w/ prefixed tokens**

Figure 4: **Explorations of outlier tokens in Llama-2-7B.** (a) Outlier token only exits in nearly 2 positions in the overall input sequence. (b) Excluding token in position 0, outlier tokens only exits in '." or "\n" tokens. (c) Outlier tokens consistently occur in the starting token (position 0) and another front but un-predictable position index. (d) Prefixing the input sequence with high-frequency outlier tokens (".\n") can constraint the outlier tokens only exit in position 0 and 1.

Figure 5: Prefixed tokens in KV cache across different models. [BOS] indicates the special token for beginning of sequence(*e.g.* "<s>" for Llama-2 and "|begin_of_text|" for Llama-3). Note that the following "_" represents space.

Model	**Prefixed token**	
	Number	**Content**
Llama-2-7B	3	.\n[BOS]
Llama-2-13B	3	the.[BOS]
Llama-2-70B	4	\n"[BOS]
Llama-3-8B(-Instruct)	1	[BOS]
Llama-3-70B(-Instruct)	3	,_[BOS]
Mistral-v0.3-7B	4	\n.to[BOS]
Qwen-2-7B	1	[BOS]

Distribution changing after setting prefixed tokens. As shown in Figure 3, prefixing outlier tokens in the KV cache significantly improves the distribution. Specifically, the $\max(\frac{\text{top-1}}{\text{median}})$ ratio of the down_proj inputs decreases from 461 to 2.4 and the $\max(\frac{\text{median}}{\text{min-1}})$ ratio of **Q/K** decreases from > 9 to < 3.5.

4.3. Block-wise Fine-tuning

Recent studies show that block-wise fine-tuning (Shao et al., 2023; Chen et al., 2024a) improves performance by accounting for inter-layer interactions (Li et al., 2021). To further enhance the performance of quantized models, we fine-tune each transformer block sequentially using a mean squared error (MSE) loss. Specifically, we introduce trainable parameters for the activation quantizer to balance the rounding and clipping errors in quantization. For dynamic activation quantization in PrefixQuant-O1, we set the tensor-wise clipping factors as trainable. Note that the clipping factors cannot be token-wise, as long-context scenarios introduce excessive storage overhead with token-wise clipping factors. For static activation quantization in PrefixQuant-O2, the quantization parameters (scaling factors and zero-points) are inherently trainable. For weight quantization, we follow the approach of EfficientQAT (Chen et al., 2024a), enabling the training of all weights and weight quantization parameters.

5. Experiments

5.1. Setups

Baseline. PrefixQuant is a versatile method applicable to any precision. We conduct experiments on two mainstream precisions: W4A8KV4, and W4A4KV4. The detailed quantization settings are illustrated in Table 1. PrefixQuant-O1 is consistent with existing methods for fair comparisons, and PrefixQuant-O2 targets to push the limitation of more efficient static quantization. We compare PrefixQuant with QuaRot (Ashkboos et al., 2024b), Atom (Zhao et al., 2023), DuQuant (Lin et al., 2024a), QoQ (Lin et al., 2024b), and SpinQuant (Liu et al., 2024b). Following QoQ, we reproduce all these methods except SpinQuant with Pile (Gao et al., 2020) calibration dataset to avoid over-fitting for fair comparisons. The detailed quantization configuration and results sources of these comparison methods can be found at Sec. A.

Models and datasets. We evaluate PrefixQuant on the Llama-2, Llama-3, Llama-3-Instruct families, Mistral-7B-v0.3, and Qwen-2-7B models. Following previous literature (Shao et al., 2023; Lin et al., 2024b), we assess PrefixQuant quantized models on language modeling and zero-shot tasks. Specifically, we evaluate on Wiki-

Table 2: **W4A4KV4 results on Llama models.** "PPL" indicates WikiText2 perplexity measured with context length 2048. "Acc." indicates the average zero-shot accuracy on 5 common-sense reasoning tasks. Grayed results use WikiText2 as calibaration dataset.

Model	Method	PPL	Acc.
2-7B	FP16	5.47	69.04
	Atom	6.12	59.73
	QuaRot	6.19	64.69
	DuQuant	6.20	66.25
	SpinQuant	5.95	65.35
	PrefixQuant-O1	**5.93**	**66.74**
	PrefixQuant-O2	6.01	66.37
2-13B	FP16	4.88	71.73
	Atom	5.31	63.51
	QuaRot	5.45	69.01
	DuQuant	5.39	69.13
	SpinQuant	5.24	69.24
	PrefixQuant-O1	**5.24**	70.05
	PrefixQuant-O2	5.32	**70.36**
2-70B	FP16	3.32	76.72
	Atom	3.73	67.52
	QuaRot	3.83	75.43
	DuQuant	3.77	74.75
	SpinQuant	3.70	75.19
	PrefixQuant-O1	**3.62**	**76.23**
	PrefixQuant-O2	3.81	75.48
3-8B	FP16	6.14	72.71
	Atom	7.76	
	QuaRot	8.41	65.15
	DuQuant	8.14	67.13
	SpinQuant	7.36	68.23
	PrefixQuant-O1	**7.26**	**71.31**
	PrefixQuant-O2	7.43	71.08
3-70B	FP16	2.85	80.03
	QuaRot	6.82	68.39
	DuQuant	5.67	74.89
	PrefixQuant-O1	**4.16**	77.08
	PrefixQuant-O2	4.41	**77.18**

Text2 (Merity et al., 2016) with a 2048 context length for perplexity, and on 5 zero-shot reasoning tasks, including PIQA (Bisk et al., 2020), ARC (Clark et al., 2018), HellaSwag (Zellers et al., 2019), and WinoGrande (Sakaguchi et al., 2021). We also test models on more challenge zero-shot MMLU (Hendrycks et al., 2020). All accuracy are measured through `lm_eval v0.4.2` (Gao et al., 2024). For accuracy, we report `acc` for WinoGrande and `acc_norm` for HellaSwag, Arc_Challenge, Arc_Easy, and PIQA, following Qserve (Lin et al., 2024b).

Grid Search Initialization Setting. We initialize the quantization parameters through grid search on 8 Pile (Gao et al., 2020) samples with a 1024 sequence length. We minimize the layer outputs for fine-grained quantization (per-channel/per-head) and block outputs for per-tensor quantization.

Fine-Tuning Setting. During fine-tuning, we optimize block output mean square error following existing works (Shao et al., 2023; Chen et al., 2024a). The dataset

Table 3: **W4A8KV4 results on Llama models.** Refer Table 2 for the metric setting.

Model	Method	PPL	Acc.
2-7B	FP16	5.47	69.04
	QoQ	5.75	67.22
	QuaRot	5.73	67.11
	PrefixQuant-O1	**5.67**	68.04
	PrefixQuant-O2	5.68	**68.09**
2-13B	FP16	4.88	71.73
	QoQ	5.12	70.56
	QuaRot	5.07	69.96
	PrefixQuant-O1	**5.05**	**71.25**
	PrefixQuant-O2	5.07	**71.25**
2-70B	FP16	3.32	76.72
	QoQ	3.52	75.91
	QuaRot	3.46	76.31
	PrefixQuant-O1	**3.44**	**76.82**
	PrefixQuant-O2	3.50	76.50
3-8B	FP16	6.14	72.71
	QoQ	6.89	71.35
	QuaRot	6.80	71.68
	PrefixQuant-O1	**6.59**	**72.57**
	PrefixQuant-O2	6.62	72.46
3-70B	FP16	2.85	80.03
	QoQ	4.36	78.12
	QuaRot	3.73	78.52
	PrefixQuant-O1	**3.37**	78.50
	PrefixQuant-O2	3.43	**78.70**

Table 4: MMLU average accuracy (zero-shot) on Llama-3-8B.

Method	Precision	MMLU Acc.
-	FP16	62.07
QuaRot	W4A4KV4	34.25
DuQuant	W4A4KV4	50.77
SpinQuant	W4A4KV4	51.93
PrefixQuant-O1	W4A4KV4	**56.00**
PrefixQuant-O2	W4A4KV4	54.65
QuaRot	W4A8KV4	38.37
DuQuant	W4A8KV4	58.01
SpinQuant	W4A8KV4	58.25
PrefixQuant-O1	W4A8KV4	**60.49**
PrefixQuant-O2	W4A8KV4	59.20

for fine-tuning consists of 512 samples from Pile with a 1024 context length. The learning rates for quantization parameters (step sizes) and full-precision weights are set to 5e-5 and 5e-6, respectively, and to 2e-5 and 2e-6 for Llama-3-70B(-Instruct) models. The fine-tuning batch size is set to 4, and the number of epochs is set to 10 for W4A8KV4 and 20 for W4A4KV4.

5.2. Comparison Results

Results on W4A4KV4. Table 2 presents the comparison results for W4A4KV4. PrefixQuant consistently outperforms existing methods. For example, under the same dynamic quantization setting on Llama-3-8B, PrefixQuant-O1 achieves a 1.12 WikiText perplexity improvement and

Table 5: Ablation study on quantization techniques used in PrefixQuant. The model used here is Llama-3-8B, and WikiText2 perplexity with 2048 context length is reported. Both PrefixQuant-O1 and PrefixQuant-O2 are start from the "Base".

	Method	Activation Quant.	W4A8KV4	W4A4KV4
	QuaRot	dynamic	6.75	8.33
Base	RTN	dynamic	12.66	1282.34
	+ rotation	dynamic	10.88	24.98
	+ Grid search initialization	dynamic	9.08	11.70
PrefixQuant-O1	**+ prefixed outliers**	**dynamic**	**6.81**	**7.53**
	+ block-wise fine-tuning	**dynamic**	**6.59**	**7.23**
PrefixQuant-O2	+ static quantization	static	29.07	141.02
	+ prefixed outliers	**static**	**6.90**	**7.93**
	+ block-wise fine-tuning	**static**	**6.62**	**7.41**

$+4.18$ points accuracy over DuQuant. Additionally, the more efficient PrefixQuant-O2 for static quantization also surpasses DuQuant, with a 0.71 perplexity improvement and $+3.95$ points accuracy.

Results on W4A8KV8. Table 3 shows the comparison results for W4A8KV8. PrefixQuant-O1 and PrefixQuant-O2 outperform both QoQ and QuaRot across most models. For instance, PrefixQuant-O1 surpasses QoQ (Lin et al., 2024b) by 0.31 perplexity and $+1.22$ points accuracy on Llama-3-8B. Similarly, PrefixQuant-O2 maintains performance benefits with a 0.28 perplexity improvement and $+1.11$ points accuracy.

Results on more models. The results in Table 15 demonstrate that PrefixQuant consistently achieves excellent performance on other models such as Mistral-7b-v0.3 and Qwen-2-7B, as well as instruction-tuned models like Llama-3-{7B,70B}-Instruct.

Results on MMLU. Table 4 presents the comparison results on zero-shot MMLU using Llama-3-8B. PrefixQuant-O1 and PrefixQuant-O2 outperform SpinQuant by $+2.24$ and $+0.95$ accuracy, respectively, in W4A8KV4 quantization. The performance advantage is even more pronounced in W4A4KV4 quantization, with improvements of $+4.07$ and $+2.72$ accuracy, respectively.

5.3. Ablation Studies

We analyze the effects of various quantization techniques implemented in PrefixQuant. These techniques are applied incrementally, and the WikiText2 perplexity results are presented in Table 5. The analysis begins with round-to-nearest (RTN) quantization on Llama-3-8B, incorporating Hadamard rotation and grid search initialization. Using W4A4KV4 as an example, we observe that introducing prefixed outliers significantly improves performance. Specifically, perplexity decreases from 11.70 to 7.53 for PrefixQuant-O1 and from 141.02 to 7.93 for PrefixQuant-O2. These improvements result not only from mitigating information loss caused by outlier tokens but also from enabling more accurate quantization parameter selection during grid searches initialization by isolating extremely large outliers (e.g., values exceeding $1e3$) in activations. Additionally, block-wise fine-tuning further enhances performance, reducing perplexity by 0.30 for PrefixQuant-O1 and by 0.52 for PrefixQuant-O2 in W4A4KV4 quantization. Additional ablation results, including analyses of the training dataset, training epochs, dynamic quantization, the number of prefixed tokens, and the content of prefixed tokens, are provided in Sec. D of the Appendix.

Figure 6: Inference speedup of W4A4 Llama-2-7B model over the FP16 model on RTX 3090 GPU. For prefilling, we report the latency to deal with 2048 input tokens. For decoding, we report the token generation speed of generate 256 new tokens with 2048 prefillinng length.

Method	prefilling (ms)	decoding (token/s)
FP16	489	43
PrefixQuant-O1 (W4A4)	183 (**2.67x**)	91 (**2.11x**)
PrefixQuant-O2 (W4A4)	178 (**2.74x**)	93 (**2.16x**)

5.4. Inference Speed

In this section, we evaluate the end-to-end inference speed of PrefixQuant under the W4A4 quantization scenario. KV quantization is not considered because it reduces memory usage at the cost of increased computation overhead and only provides speedup with large batch sizes (Liu et al., 2024a). As shown in Table 6, PrefixQuant achieves an approximate $2.7\times$ speedup in prefilling and a $2.1\times$ speedup in decoding compared to the FP16 model.

6. Conclusion

In this paper, we propose PrefixQuant, which provides a comprehensive exploration of outlier tokens and introduces an efficient and effective method to handle them by prefixing these tokens in the KV cache. Additionally, we design new trainable parameters for activation quantization to minimize quantization error. The proposed PrefixQuant method achieves excellent performance across various models, quan-

tization precisions, and granularities. The simplicity and broad applicability of PrefixQuant make it a promising direction for future research on LLM compression and optimization.

7. Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Ashkboos, S., Markov, I., Frantar, E., Zhong, T., Wang, X., Ren, J., Hoefler, T., and Alistarh, D. Towards end-to-end 4-bit inference on generative large language models. *arXiv preprint arXiv:2310.09259*, 2023.

Ashkboos, S., Croci, M. L., Nascimento, M. G. d., Hoefler, T., and Hensman, J. Slicegpt: Compress large language models by deleting rows and columns. *arXiv preprint arXiv:2401.15024*, 2024a.

Ashkboos, S., Mohtashami, A., Croci, M. L., Li, B., Jaggi, M., Alistarh, D., Hoefler, T., and Hensman, J. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456*, 2024b.

Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, pp. 7432–7439, 2020.

Bondarenko, Y., Nagel, M., and Blankevoort, T. Quantizable transformers: Removing outliers by helping attention heads do nothing. *Advances in Neural Information Processing Systems*, 36, 2024.

Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y., Lundberg, S., et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.

Chen, M., Shao, W., Xu, P., Wang, J., Gao, P., Zhang, K., Qiao, Y., and Luo, P. Efficientqat: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062*, 2024a.

Chen, T., Li, Z., Xu, W., Zhu, Z., Li, D., Tian, L., Barsoum, E., Wang, P., and Cheng, J. Ternaryllm: Ternarized large language model. *arXiv preprint arXiv:2406.07177*, 2024b.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

Computer, T. Redpajama: an open dataset for training large language models, 2023. URL https://github.com/togethercomputer/RedPajama-Data.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems*, 35:30318–30332, 2022.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation, 07 2024. URL https://zenodo.org/records/12608602.

Gu, X., Pang, T., Du, C., Liu, Q., Zhang, F., Du, C., Wang, Y., and Lin, M. When attention sink emerges in language models: An empirical view. *arXiv preprint arXiv:2410.10781*, 2024.

Han, C., Wang, Q., Xiong, W., Chen, Y., Ji, H., and Wang, S. Lm-infinite: Simple on-the-fly length generalization for large language models. *arXiv preprint arXiv:2308.16137*, 2023.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Li, Y., Gong, R., Tan, X., Yang, Y., Hu, P., Zhang, Q., Yu, F., Wang, W., and Gu, S. Brecq: Pushing the limit of post-training quantization by block reconstruction. *arXiv preprint arXiv:2102.05426*, 2021.

Lin, H., Xu, H., Wu, Y., Cui, J., Zhang, Y., Mou, L., Song, L., Sun, Z., and Wei, Y. Duquant: Distributing outliers via dual transformation makes stronger quantized llms. *arXiv preprint arXiv:2406.01721*, 2024a.

Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., and Han, S. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., and Han, S. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532*, 2024b.

Liu, J., Gong, R., Wei, X., Dong, Z., Cai, J., and Zhuang, B. Qllm: Accurate and efficient low-bitwidth quantization for large language models. *arXiv preprint arXiv:2310.08041*, 2023.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024a.

Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. Spinquant–llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024b.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., Van Baalen, M., and Blankevoort, T. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

Sakaguchi, K., Bras, R. L., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

Shao, W., Chen, M., Zhang, Z., Xu, P., Zhao, L., Li, Z., Zhang, K., Gao, P., Qiao, Y., and Luo, P. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*, 2023.

Son, S., Park, W., Han, W., Kim, K., and Lee, J. Prefixing attention sinks can mitigate activation outliers for large language model quantization. *arXiv preprint arXiv:2406.12016*, 2024.

Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Sun, M., Chen, X., Kolter, J. Z., and Liu, Z. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Wei, X., Zhang, Y., Zhang, X., Gong, R., Zhang, S., Zhang, Q., Yu, F., and Liu, X. Outlier suppression: Pushing the limit of low-bit transformer language models. *Advances in Neural Information Processing Systems*, 35:17402–17414, 2022.

Wei, X., Zhang, Y., Li, Y., Zhang, X., Gong, R., Guo, J., and Liu, X. Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling. *arXiv preprint arXiv:2304.09145*, 2023a.

Wei, X., Zhang, Y., Li, Y., Zhang, X., Gong, R., Guo, J., and Liu, X. Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling. *arXiv preprint arXiv:2304.09145*, 2023b.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pp. 38087–38099. PMLR, 2023a.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023b.

Yang, J., Kim, H., and Kim, Y. Mitigating quantization errors due to activation spikes in glu-based llms. *arXiv preprint arXiv:2405.14428*, 2024.

Yuan, Z., Shang, Y., Zhou, Y., Dong, Z., Xue, C., Wu, B., Li, Z., Gu, Q., Lee, Y. J., Yan, Y., et al. Llm inference unveiled: Survey and roofline model insights. *arXiv preprint arXiv:2402.16363*, 2024.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

Zhao, Y., Lin, C.-Y., Zhu, K., Ye, Z., Chen, L., Zheng, S., Ceze, L., Krishnamurthy, A., Chen, T., and Kasikci, B. Atom: Low-bit quantization for efficient and accurate llm serving. *arXiv preprint arXiv:2310.19102*, 2023.

Overview of Appendix

We detailed the content of Appendix here:

- Sec A details results sources of comparison methods.
- Sec. B illustrates the detailed image of hadamaed rotation within a transformer block.
- Sec. C details the quantization time of PrefixQuant.
- Sec. D gives more ablation studies of PrefixQuant, including the fine-tuning dataset, training epoch, and number of prefixed tokens.
- Sec. E offers the comparison results in long-context scnarios.
- Sec. F demonstrates that proposed PrefixQuant can also play as a plug-in to enhance the performance of existing weight-only quantization methods.
- Sec. G presents the detailed accuracy number of each zero-shot task, and provide more results of PrefixQuant on Mistral-v0.3-7B, Qwen-2-7B, and Llama-3-{8B,70B}-Instruct.
- Sec. H illustrate more visualization of inputs of linear layer and **Q/K/V** on more models, including Llama-3-{8B,70B}, Mistral-7B-v0.3, Qwen-2-7B.

A. Results Sources of Comparison Methods

We compare our proposed PrefixQuant with several other methods: QuaRot (Ashkboos et al., 2024b), Atom (Zhao et al., 2023), QoQ (Lin et al., 2024b), SmoothQuant (Xiao et al., 2023a), SpinQuant (Liu et al., 2024b), and EfficientQAT (Chen et al., 2024a). The data for our comparisons either come directly from the official publications of these methods, from other papers, or from our own reproduction of the methods. The source of the data for each method is outlined as follows:

- **QuaRot**: We present the performance of QuaRot using the Pile calibration dataset. The results for Llama-2 models with W4A4KV4 come from QoQ (Lin et al., 2024b), while the rest are reproduced using the official open-source code.
- **DuQuant**: We reproduce DuQuant with Pild calibration dataset through their official open-source code. Note that we change the evaluation toolbox to lm-eval v0.4.2 for more accurate evaluation.
- **Atom**: We present the performance of Atom using the Pile calibration dataset. The results are sourced from QoQ (Lin et al., 2024b).
- **QoQ**: We present the performance of QoQ using the Pile calibration dataset. The results for Llama-2 come from QoQ (Lin et al., 2024b), and the Llama-3 results are reproduced using the official open-source code.
- **SmoothQuant**: We present the performance of SmoothQuant using the Pile calibration dataset. All results are reproduced using the open-source code from QoQ (Lin et al., 2024b).
- **SpinQuant**: All results are reproduced using the official open-source code and the pre-trained rotation matrix. Note that SpinQuant directly trains on the WikiText2 dataset.
- **EfficientQAT**: All results are reproduced using the official open-source code and the pre-quantized models.

B. Details of Rotation

Hadamard rotation (Ashkboos et al., 2024b; Liu et al., 2024b) redistributes outlier channels across all channels, achieving uniform distribution within each token. The Hadamard matrix $\mathbf{H}$ is an orthogonal matrix with $\mathbf{H}\mathbf{H}^T = \mathbf{I}$, and its entries are $\{+1, -1\}$ at the same scale. Hadamard rotation can be applied to all activations and use inverse rotation on corresponding weights to maintain computational invariance (Ashkboos et al., 2024a). Specifically, the rotation includes absorbable and

online rotations. As shown in Figure 7, we follow SpinQuant (Liu et al., 2024b) to set $R1$, $R2$, $R3$ and $R4$ rotations, details as follows.

Absorbable Rotation. Hadamard rotation of activation can be absorbed into the previous linear layer if there is no intervening non-linear operation. Thus, the rotation of input activations for q/k/v/gate/up_proj (R_1) and head-wise rotation for o_proj input activations (R_2) can be fully absorbed without adding computation during inference.

Online Rotation. Some rotations must be executed online, including output activations of q_proj and k_proj after RoPE (Su et al., 2024) (R_3), and the input activation of down_proj (R_4). These online rotations are efficiently implemented using the Walsh-Hadamard transform without significant overhead.

If not specifically mentioned, we activate all rotation (R_1, R_2, R_3 and R_4) in weight-activation quantization scenes, and only activate absorbable rotation (R_1 and R_2) in weight-only quantization.

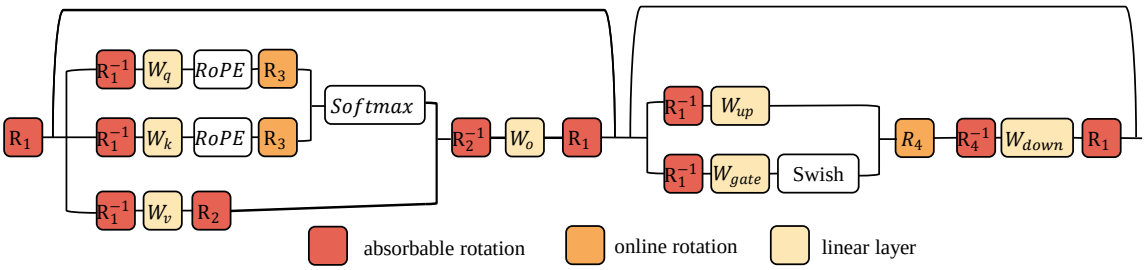


Figure 7: Illustrate of hadamard rotation within a transformer block of Llama (Touvron et al., 2023) model.

C. Quantization Time

Table 6 shows the quantization time for PrefixQuant. PrefixQuant identifies prefixed tokens quickly, taking only 0.2 minutes for Llama-3-8B and 1 minute for Llama-3-70B. In contrast, the recent CushionCache (Son et al., 2024) requires 12 hours for the same task on Llama-3-8B. Additionally, the grid-search initialization is efficient, taking 0.7 minutes for Llama-3-8B and 12 minutes for Llama-3-70B. Experiments in Tables 2 and 3 demonstrate that PrefixQuant, even without fine-tuning, outperforms previous methods (Lin et al., 2024b; Ashkboos et al., 2024b). Fine-tuning requires more time, taking 2.2 hours for Llama-3-8B and 17 hours for Llama-3-70B, but it can successfully enhances the potential of low-bit quantization.

Table 6: The quantization time of PrefixQuant on single NVIDIA-A100-80GB GPU. Fine-tuning indicates the time of 20 fine-tuning epochs of W4A4KV4.

Model	Find Prefixed Outliers	Grid-search initialization	Fine-tuning
Llama-3-8B	0.2 m	0.7 m	2.2 h
Llama-3-70B	1 m	12 m	17 h

D. More Ablation Results

Fine-tuning Datasets. Table 7a shows results with different fine-tuning datasets, including C4 (Raffel et al., 2020), RedPajama (Computer, 2023), and Pile (Gao et al., 2020). We find that Pile achieves the best performance. Additionally, we ablate the sequence length of each training sample and the total training tokens. Table 7b shows that a sequence length of 1024 achieves the best performance. Table 7c demonstrates that fine-tuning on 512×1024 tokens achieves satisfactory performance, with further increases in training samples only marginally improving performance. Note that the optimal token number for fine-tuning datasets may change with quantization precision. Generally, lower precision requires more training data. For example, EfficientQAT shows that 4096×2048 tokens are needed for W2A16 quantization, while our paper shows that only 512×1024 tokens are needed for W4A4 quantization.

Training Epochs. Table 8 demonstrates that 10 and 20 epochs are sufficient for the convergence of fine-tuning on W4A8KV4 and W4A4KV4.

Table 7: Ablation studies on calibration dataset, including (a) Dataset type, (b) Training sequence length and (c) Total training tokens. "N" indicates number of training samples, and "S" is the length of each samples. The model used here is Llama-3-8B with W4A4KV4 (PrefixQuant-O2) quantization. Our default settings are marked in gray.

(a) Dataset

Dataset	Wiki PPL
C4	7.60
RedPajama	7.49
Pile	**7.42**

(b) Sequence length

N × S	Wiki PPL
256 ×2048	7.65
512×1024	**7.42**
1024×512	7.65

(c) Total token number

N × S	Wiki PPL
256 ×1024	7.46
512 ×1024	7.42
1024×1024	**7.41**

Table 8: Ablation study about training epochs. The model used here is Llama-3-8B with PrefixQuant-O2, and WikiText2 perplexity with 2048 context length is reported. Our default settings are marked in gray.

Epochs	W4A8KV4	W4A4KV4
0 (w/o FT)	6.90	7.93
5	6.66	7.53
10	**6.63**	7.47
20	6.63	7.42
30	6.63	**7.41**

Table 9: Ablation study about the number of prefixed tokens. WikiText2 perplexity with 2048 context length and W4A4KV4 (PrefixQuant-O2) quantization is reported. Number n indicates the first n tokens in Table 5 are set as the prefixed tokens.

Model	Method	0	1	2	3	4
Llama-2-7B	PrefixQuant w/o FT	333.52	74.37	6.21	6.22	-
Llama-2-7B	PrefixQuant	17.63	10.71	**6.01**	**6.01**	
Mistral-7B-v0.3	PrefixQuant w/o FT	90.02	6.12	5.84	6.43	5.89
Mistral-7B-v0.3	PrefixQuant	15.97	7.08	5.83	5.95	**5.79**

Table 10: Ablation study about the content of prefixed tokens. WikiText2 perplexity with 2048 context length and W4A4KV4 (PrefixQuant-O2) quantization is reported. "default" refers to the prefixed tokens obtained through the proposed method. "random" represents the average performance of 10 times with randomly selected prefixed tokens.

Model	Type	Prefixed	Wiki PPL (PrefixQuant w/o FT)
Llama-2-7B	default	.\n[BOS]	6.22
Llama-2-7B	only highest frequency	...	12.07
Llama-2-7B	random	-	66.51
Mistral-7B-v0.3	default	\n.to[BOS]	5.89
Mistral-7B-v0.3	only highest frequency	\n\n\n\n	6.23
Mistral-7B-v0.3	random	-	80.05

Number of Prefixed Tokens. In Sec. 4.2, we determine the number of prefixed tokens by calculating the average number of outlier tokens and adding an additional [BOS] token. Table 5 illustrates the specific number and content of these tokens. We use Llama-2-7B (3 outlier tokens) and Mistral-7B-v0.3 (4 outlier tokens) to study the impact of the number of prefixed tokens. Table 9 shows that the adaptively calculated number of prefixed tokens achieves the best performance. Notably, for models like Llama-2-7B, using 2 prefixed tokens without the additional [BOS] token also yields excellent performance. For consistency and simplicity, we include the [BOS] token in the prefixed tokens in our experiments.

Content of Prefixed Tokens. PrefixQuant determines the number of outlier tokens N and designates the top-N high-frequency outlier tokens as prefixes in the KV cache. Table 10 examines various prefixed tokens with the same token count.

Table 11: Comparisons in long-context scenario of Llama-3-8B. We report the WikiText2 perplexity with context length 8192. We do not report SpinQuant results because it overfits to WikiText2 datasets.

Method	Precision	PPL.
-	FP16	5.54
QuaRot	W4A8KV4	6.79
DuQuant	W4A8KV4	6.19
PrefixQuant-O1	W4A8KV4	**5.94**
PrefixQuant-O2	W4A8KV4	6.04
QuaRot	W4A4KV4	8.41
DuQuant	W4A4KV4	7.27
PrefixQuant-O1	W4A4KV4	**6.58**
PrefixQuant-O2	W4A4KV4	6.82

The results show that using the top-N high-frequency tokens as prefixed tokens significantly outperforms using only the highest-frequency or randomly selected tokens.

E. Comparisons in Long-Context Scenarios

PrefixQuant-O1 uses a shared clipping factor for each layer, while PrefixQuant-O2 further shares a scaling factor across the entire tensor. A longer input context implies that more activations share the same clipping or scaling factor. This raises a concern about whether PrefixQuant remains effective in long-context scenarios. Table 11 shows that PrefixQuant consistently outperforms existing methods at a context length of 8192, demonstrating the strong generalization ability of the proposed method.

Table 12: Weight-only quantization results. "g" indicates group size for weight quantization. EfficientQAT only execute Block-AP and without E2E-QP for the fair comparisons in block-wise reconstruction scenario. We providing WikiText2 perplexity with 2048 context length and detailed zero-shot accuracy of weight-only quantization by `lm_eval v0.4.2`. We report `acc` for WinoGrande and `acc_norm` for HellaSwag, ArcC, ArcE, and PIQA.

Model	Method	Precision	Wiki PPL	WinoGrande	HellaSwag	ArcC	ArcE	PiQA	Avg. Acc.
3-8B	Baseline	FP16	6.14	72.61	79.17	53.41	77.69	80.69	72.71
	EfficientQAT	W3A16g128	7.34	70.48	75.09	51.37	77.9	79.16	70.80
	PrefixQuant	W3A16g128	7.17	72.38	76.54	52.65	78.37	80.58	72.10
	EfficientQAT	W2A16g128	13.55	62.04	62.49	36.6	60.44	73.18	58.95
	PrefixQuant	W2A16g128	11.97	66.22	66.54	41.81	69.61	75.84	64.00
3-70B	Baseline	FP16	2.85	80.51	84.9	64.33	85.9	84.49	80.03
	EfficientQAT	W3A16g128	4.89	78.77	83.74	55.03	78.66	82.05	75.65
	PrefixQuant	W3A16g128	4.79	78.22	84.03	60.15	83.00	83.35	77.75
	EfficientQAT	W2A16g128	16.79	66.14	73.01	48.21	73.57	78.45	67.88
	PrefixQuant	W2A16g128	11.01	72.3	78.55	53.67	77.9	80.63	72.61

F. Extend to Weight-only Quantization

In addition to static activation quantization, setting prefixed outliers in the KV-cache improves training stability (Chen et al., 2024b) and reduces information loss from outlier tokens, can also enhancing weight-only quantization performance. To verify this, we compare PrefixQuant with the recent state-of-the-art weight-only quantization method, EfficientQAT (Chen et al., 2024a), in a block-wise fine-tuning scenario. Following EfficientQAT, we use 4096 RedPajama (Computer, 2023) with a 2048 context length to train for 2 epochs. The learning rates for quantization parameters and full-precision weights are set to 5e-5 and 5e-6, except for W2A16g128 Llama-3-8B, where they are 1e-4 and 2e-5, respectively. As shown in Table 12, PrefixQuant significantly surpasses EfficientQAT with $+5.05$ and $+4.73$ points in average accuracy on W2A16g128 Llama-3-8B and Llama-3-70B, respectively.

G. Full Results of Weight-Activation quantization

Table 13: W8A8 performance comparisons with other methods that also set prefixed tokens in KV cache.

Model	Method	Activation Quant	Wiki PPL
2-7B	QFeP	per-tensor dynamic	5.75
	CushionCache	per-tensor static	5.87
	PrefixQuant-O2	per-tensor static	**5.48**
2-13B	QFeP	per-tensor dynamic	6.00
	PrefixQuant-O2	per-tensor static	**4.89**
2-70B	QFeP	per-tensor dynamic	6.01
	PrefixQuant-O2	per-tensor static	**3.39**
3-8B	CushionCache	per-tensor static	7.37
	PrefixQuant-O2	per-tensor static	**6.17**

G.1. Comparisons with Related Works

CushionCache (Son et al., 2024) and QFeP (Yang et al., 2024) also set prefixed tokens in the KV cache to reduce outliers. However, they experience significant performance degradation even with W8A8 quantization. Table 13 shows that PrefixQuant outperforms QFeP by 2.62 perplexity on Llama-2-70B and surpasses CushionCache by 1.20 perplexity on Llama-3-8B.

G.2. Detailed Accuracy Results

In the main paper, we present the average accuracy of five common reasoning tasks for brevity. Here, we provide detailed results for each task in Table 14.

G.3. Results on More Models

Table 15 shows the effectiveness of the proposed PrefixQuant in other models, including Mistral-v0.3-7B and Qwen-2-7B. It also includes instruction-tuned models such as Llama-3-{8B,70B}-Instruct.

H. More Visualizations

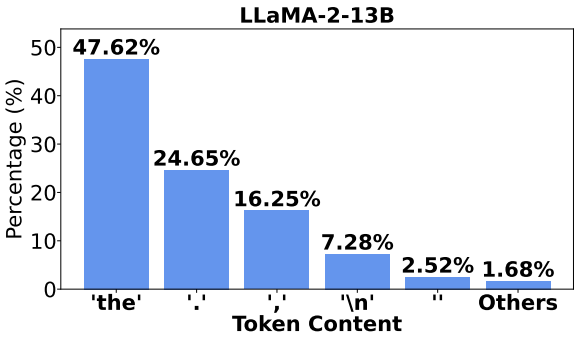
H.1. Outlier Token

In Figure 8, we showcase the four most frequently occurring outlier tokens in Llama-2-{13B,70B}, Llama-3-70B, and Mistral-7B-v0.3. Specifically, Table 5 selects the top-o high-frequent outlier tokens as the prefixed tokens. It is important to note that we do not visualize the outlier tokens in Llama-3-8B and Qwen-2-7B because all the outlier tokens in these two models appear in the initial tokens.

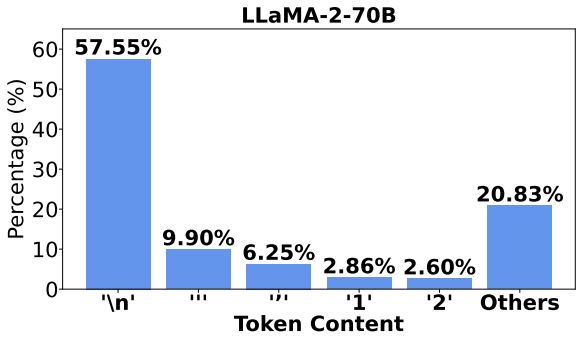
H.2. Magnitude Distribution

We illustrate more token-wise maximum values distribution of other models. Details are as follows:

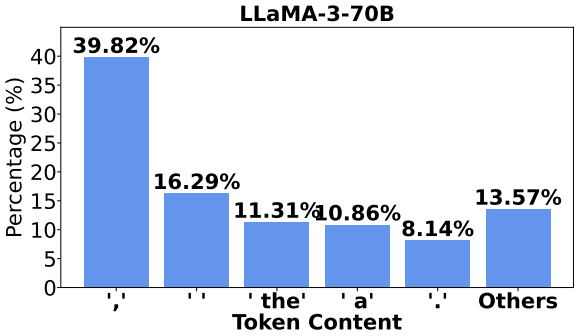
- **Llama-2-7B**: Figure 9 and Figure 10 illustrate the distribution of input activation and **Q/K/V**, respectively.
- **Llama-2-13B**: Figure 11 and Figure 12 illustrate the distribution of input activation and **Q/K/V**, respectively.
- **Llama-3-8B**: Figure 13 and Figure 14 illustrate the distribution of input activation and **Q/K/V**, respectively.
- **Llama-3-70B**: Figure 15 and Figure 16 illustrate the distribution of input activation and **Q/K/V**, respectively.
- **Qwen-2-7B**: Figure 17 and Figure 18 illustrate the distribution of input activation and **Q/K/V**, respectively.
- **Mistral-7B-v0.3**: Figure 19 and Figure 20 illustrate the distribution of input activation and **Q/K/V**, respectively.



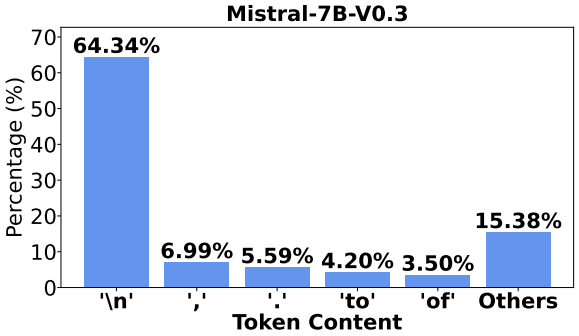
(a) Llama-2-13B



(b) Llama-2-70B

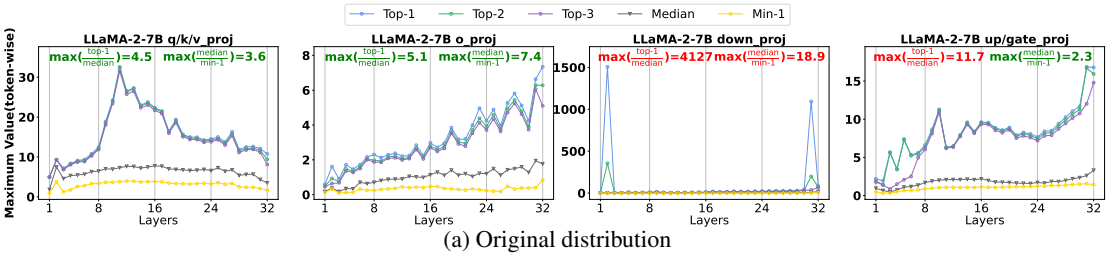


(c) Llama-3-70B

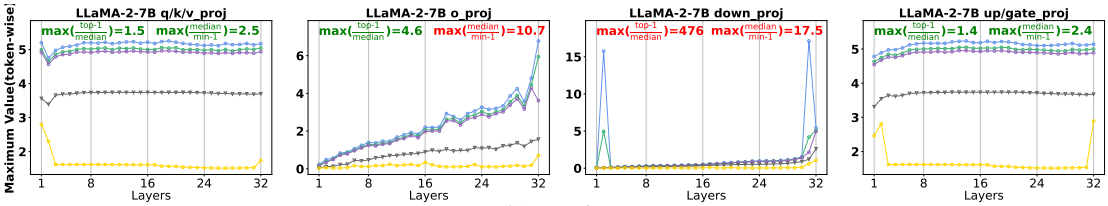


(d) Mistral-7B-v0.3

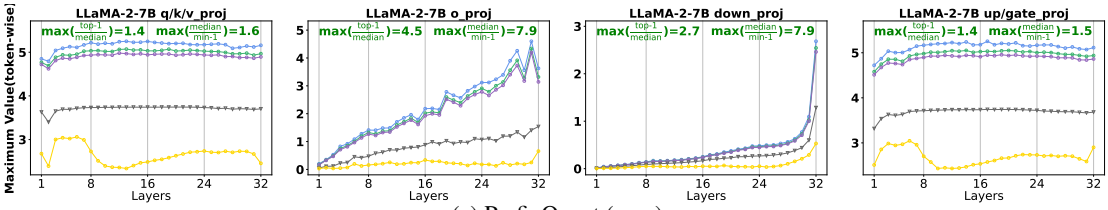
Figure 8: **Content of outlier tokens in different models.** Note that we do not count the outlier tokens situated at the initial token.



(a) Original distribution



(b) Rotation



(c) PrefixQuant (ours)

Figure 9: **Distribution of token-wise maximum values for linear layers inputs in Llama-2-7B.** Top-N indicates the N-th largest value, Min-N indicates the N-th smallest value.

Table 14: Continuation of Table 2 and Table 3, providing detailed zero-shot accuracy of weight-activation quantization of Llama models by `lm_eval v0.4.2`. We report `acc` for WinoGrande and `acc_norm` for HellaSwag, ArcC, ArcE, and PIQA.).

Model	Method	Precision	WinoGrande	HellaSwag	ArcC	ArcE	PiQA	Avg. Acc.
2-7B	Baseline	FP16	69.22	76.00	46.25	74.62	79.11	69.04
	Atom	W4A4KV4	62.75	69.37	38.40	52.99	75.14	59.73
	QuaRot	W4A4KV4	64.40	72.3	41.47	68.06	76.17	64.48
	DuQuant	W4A4KV4	67.09	72.53	43.26	71.38	76.99	66.25
	SpinQuant	W4A4KV4	66.54	73.15	41.64	69.32	76.12	65.35
	PrefixQuant-O1 w/o FT	W4A4KV4	66.85	74.27	43.86	72.35	77.97	**67.06**
	PrefixQuant-O1	W4A4KV4	67.48	73.77	43.17	71.3	77.97	66.74
	PrefixQuant-O2 w/o FT	W4A4KV4	67.80	73.75	43.94	71.51	77.2	66.84
	PrefixQuant-O2	W4A4KV4	66.54	73.42	43.09	71.17	77.64	66.37
	QoQ	W4A8KV4	68.03	74.00	43.60	72.81	77.64	67.22
	QuaRot	W4A8KV4	66.77	74.56	43.86	72.39	77.97	67.11
	PrefixQuant-O1 w/o FT	W4A8KV8	69.53	75.49	44.8	73.32	77.64	**68.16**
	PrefixQuant-O1	W4A8KV8	69.69	75.3	44.28	73.19	77.75	68.04
	PrefixQuant-O2 w/o FT	W4A8KV4	69.14	75.12	44.45	73.06	77.53	67.86
	PrefixQuant-O2	W4A8KV4	69.06	75.25	44.8	73.19	78.13	68.09
2-13B	Baseline	FP16	72.22	79.37	49.06	77.48	80.52	71.73
	Atom	W4A4KV4	67.40	73.84	42.32	57.49	76.50	63.51
	QuaRot	W4A4KV4	67.88	75.28	45.65	72.35	77.48	67.73
	DuQuant	W4A4KV4	68.9	76.65	47.7	74.24	78.18	69.13
	SpinQuant	W4A4KV4	67.88	77.01	46.76	75.97	78.56	69.24
	PrefixQuant-O1 w/o FT	W4A4KV4	72.38	76.92	47.7	75.8	79.43	**70.45**
	PrefixQuant-O1	W4A4KV4	71.98	76.3	46.59	76.43	78.94	70.05
	PrefixQuant-O2 w/o FT	W4A4KV4	72.06	76.54	46.67	75.8	78.51	69.92
	PrefixQuant-O2	W4A4KV4	72.53	76.12	47.70	76.09	79.38	70.36
	QoQ	W4A8KV4	70.96	77.80	48.38	75.97	79.71	70.56
	QuaRot	W4A8KV4	70.24	78.21	47.01	74.49	79.87	69.96
	PrefixQuant-O1 w/o FT	W4A8KV8	72.77	77.6	48.46	77.36	80.36	**71.31**
	PrefixQuant-O1	W4A8KV8	72.14	77.71	48.98	76.89	80.52	71.25
	PrefixQuant-O2 w/o FT	W4A8KV4	72.77	77.49	48.12	77.06	79.92	71.07
	PrefixQuant-O2	W4A8KV4	72.77	77.54	48.72	76.81	80.41	71.25
2-70B	Baseline	FP16	79.48	84.31	56.91	80.30	82.54	76.71
	Atom	W4A4KV4	74.27	79.06	46.08	58.25	79.92	67.52
	QuaRot	W4A4KV4	76.24	81.82	56.23	80.43	82.43	75.43
	DuQuant	W4A4KV4	75.45	81.95	55.03	79	82.32	74.75
	SpinQuant	W4A4KV4	75.85	82.36	56.31	79.17	81.61	75.19
	PrefixQuant-O1 w/o FT	W4A4KV4	77.98	81.38	55.55	78.7	81.12	74.95
	PrefixQuant-O1	W4A4KV4	78.77	83.23	56.48	79.92	82.75	**76.23**
	PrefixQuant-O2 w/o FT	W4A4KV4	75.45	80.51	52.3	77.06	81.12	73.29
	PrefixQuant-O2	W4A4KV4	77.35	82.3	56.4	79.29	82.05	75.48
	QoQ	W4A8KV4	77.51	82.78	56.83	79.80	82.64	75.91
	QuaRot	W4A8KV4	77.03	83.30	57.08	81.27	82.86	76.31
	PrefixQuant-O1 w/o FT	W4A8KV8	78.06	83.64	55.12	79.71	82.15	75.74
	PrefixQuant-O1	W4A8KV8	79.64	83.97	57.68	80.18	82.64	**76.82**
	PrefixQuant-O2 w/o FT	W4A8KV4	77.35	82.79	54.35	78.28	82.21	75.00
	PrefixQuant-O2	W4A8KV4	79.08	83.56	57.42	80.39	82.05	76.50
3-8B	Baseline	FP16	72.61	79.17	53.41	77.69	80.69	72.71
	QuaRot	W4A4KV4	65.98	72.38	44.45	67.3	75.63	65.15
	DuQuant	W4A4KV4	68.59	74.27	46.5	70.41	75.9	67.13
	SpinQuant	W4A4KV4	69.22	74.83	45.99	74.07	77.04	68.23
	PrefixQuant-O1 w/o FT	W4A4KV4	70.32	75.86	48.38	71.46	77.58	68.72
	PrefixQuant-O1	W4A4KV4	70.88	75.95	52.47	78.7	79.6	**71.52**
	PrefixQuant-O2 w/o FT	W4A4KV4	69.14	75.46	47.1	72.94	77.2	68.37
	PrefixQuant-O2	W4A4KV4	71.9	75.44	50.68	78.32	79.05	71.08
	QoQ	W4A8KV4	73.4	77.23	50.87	75.59	79.65	71.35
	QuaRot	W4A8KV4	72.74	77.35	51.62	77.48	79.22	71.68
	PrefixQuant-O1 w/o FT	W4A8KV8	71.74	77.99	50.17	74.07	79.22	70.64
	PrefixQuant-O1	W4A8KV8	73.16	77.95	52.56	79.17	80.03	**72.57**
	PrefixQuant-O2 w/o FT	W4A8KV4	71.19	77.65	48.98	73.99	79.65	70.29
	PrefixQuant-O2	W4A8KV4	72.53	77.97	52.65	79.25	79.92	72.46
3-70B	Baseline	FP16	80.51	84.9	64.33	85.9	84.49	80.03
	QuaRot	W4A4KV4	68.51	76.75	47.01	72.31	77.37	68.39
	DuQuant	W4A4KV4	70.8	79.89	59.04	82.91	81.83	74.89
	SpinQuant	W4A4KV4	76.4	80.9	56	77.3	80.8	74.28
	PrefixQuant-O1 w/o FT	W4A4KV4	77.74	83.61	58.19	80.3	82.43	76.45
	PrefixQuant-O1	W4A4KV4	77.74	84.06	58.96	81.31	83.35	77.08
	PrefixQuant-O2 w/o FT	W4A4KV4	77.43	83.48	58.87	79.88	82.32	76.40
	PrefixQuant-O2	W4A4KV4	77.35	83.79	60.15	81.31	83.3	**77.18**
	QoQ	W4A8KV4	80.11	83.7	61.01	82.79	83	78.12
	QuaRot	W4A8KV4	80.35	84.03	62.12	84.64	83.46	78.92
	PrefixQuant-O1 w/o FT	W4A8KV8	78.14	84.92	59.73	81.06	83.79	77.53
	PrefixQuant-O1	W4A8KV8	79.4	85.03	61.69	81.9	84.49	78.50
	PrefixQuant-O2 w/o FT	W4A8KV4	79.23	84.71	59.39	81.57	84.22	77.82
	PrefixQuant-O2	W4A8KV4	79.48	84.86	62.29	82.53	84.33	**78.70**

Table 15: Results of proposed PrefixQuant-O1/O2 on other models.

Model	Precision	Wiki PPL	WinoGrande	HellaSwag	ArcC	ArcE	PiQA	Avg. Acc.
Mistral-v0.3-7B	FP16	5.32	73.88	80.43	52.3	78.28	82.26	73.43
	W4A8KV4-O1	5.49	72.93	80.14	52.13	78.79	81.34	73.07
	W4A8KV4-O2	5.51	73.88	79.8	52.05	79.42	80.79	73.19
	W4A4KV4-O1	5.76	70.88	78.39	50	77.74	80.41	71.48
	W4A4KV4-O2	5.79	71.51	78.12	49.66	78.03	79.92	71.45
Qwen-2-7B	FP16	7.14	72.3	78.96	52.65	78.75	80.96	72.72
	W4A8KV4-O1	7.36	72.53	77.75	50.77	76.77	80.41	71.65
	W4A8KV4-O2	8.04	71.43	76.77	53.67	77.95	78.45	71.65
	W4A4KV4-O1	7.76	69.93	75.56	51.02	76.81	78.56	70.38
	W4A4KV4-O2	8.37	68.75	74.92	48.21	74.75	79.49	69.22
Llama-3-8B-Instruct	FP16	8.29	71.82	75.81	56.83	79.76	78.51	72.55
	W4A8KV4-O1	8.73	70.8	74.29	54.18	76.94	78.78	71.00
	W4A8KV4-O2	8.74	70.17	74.6	54.44	77.65	77.97	70.97
	W4A4KV4-O1							
	W4A4KV4-O2	8.96	69.53	74.66	52.65	76.35	76.66	69.97
Llama-3-70B-Instruct	FP16	5.33	75.69	82.58	64.42	84.97	82.15	77.96
	W4A8KV4-O1	5.82	77.11	82.42	66.72	84.47	81.72	78.49
	W4A8KV4-O2	5.96	77.74	81.97	65.87	84.93	81.56	78.41
	W4A4KV4-O1	6.49	75.77	81.51	65.96	83.8	81.28	77.66
	W4A4KV4-O2	6.80	75.93	80.64	64.76	83.88	81.23	77.29

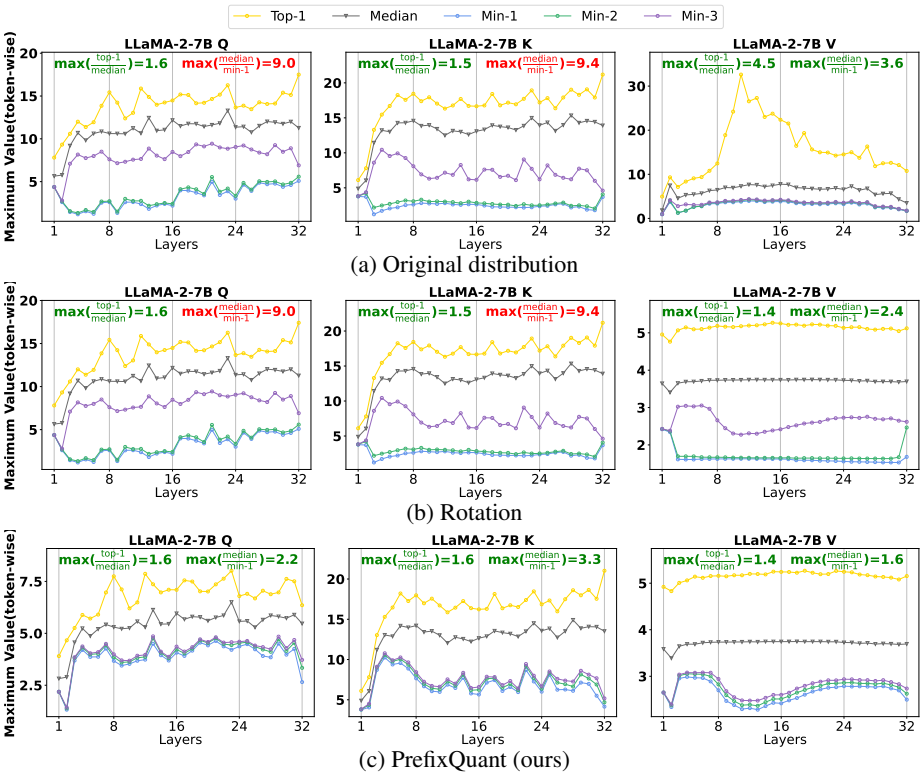


Figure 10: **Distribution of token-wise maximum values for Q/K/V in Llama-2-7B.** Same present rules as Figure 9a except that ratios greater than 5 are marked with red.

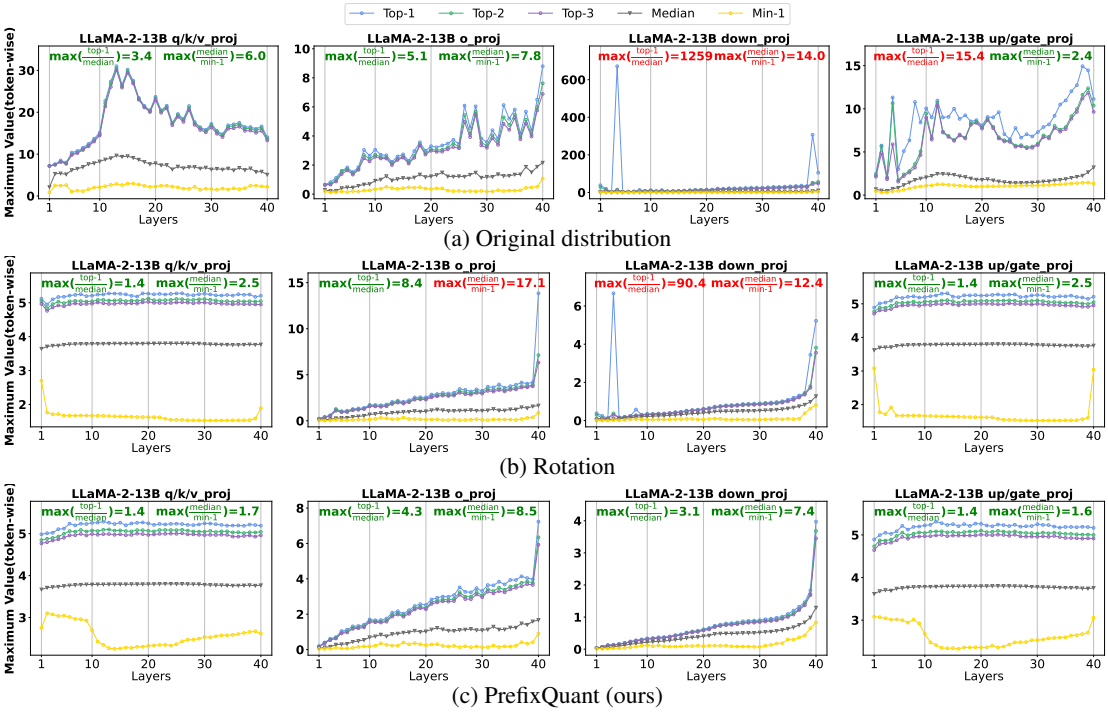


Figure 11: **Distribution of token-wise maximum values for linear layers inputs in Llama-2-13b.**

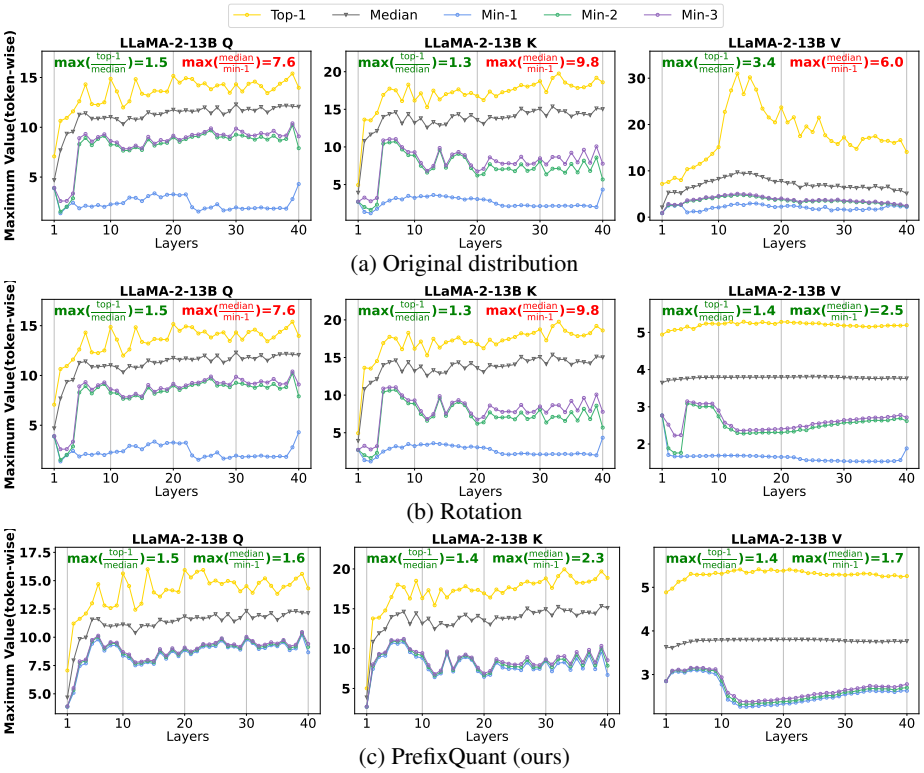


Figure 12: **Distribution of token-wise maximum values for Q/K/V in Llama-2-13b.** Same present rules as Figure 11a except that ratios greater than 5 are marked with red.

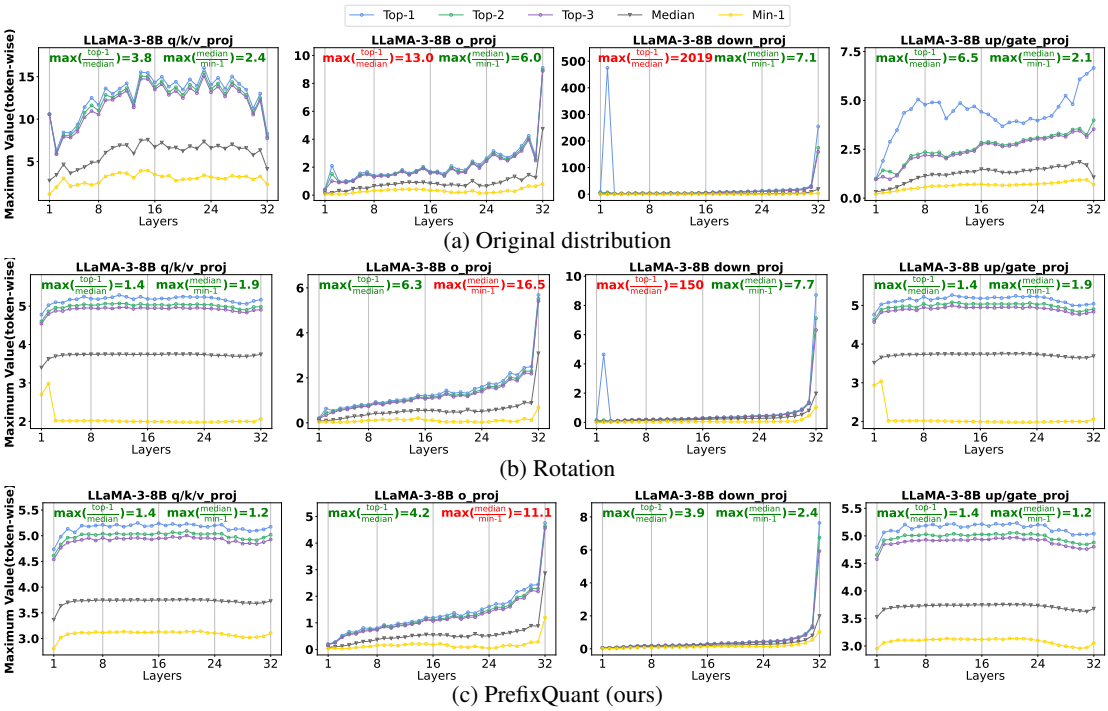


Figure 13: **Distribution of token-wise maximum values for linear layers inputs in Llama-3-8b.**

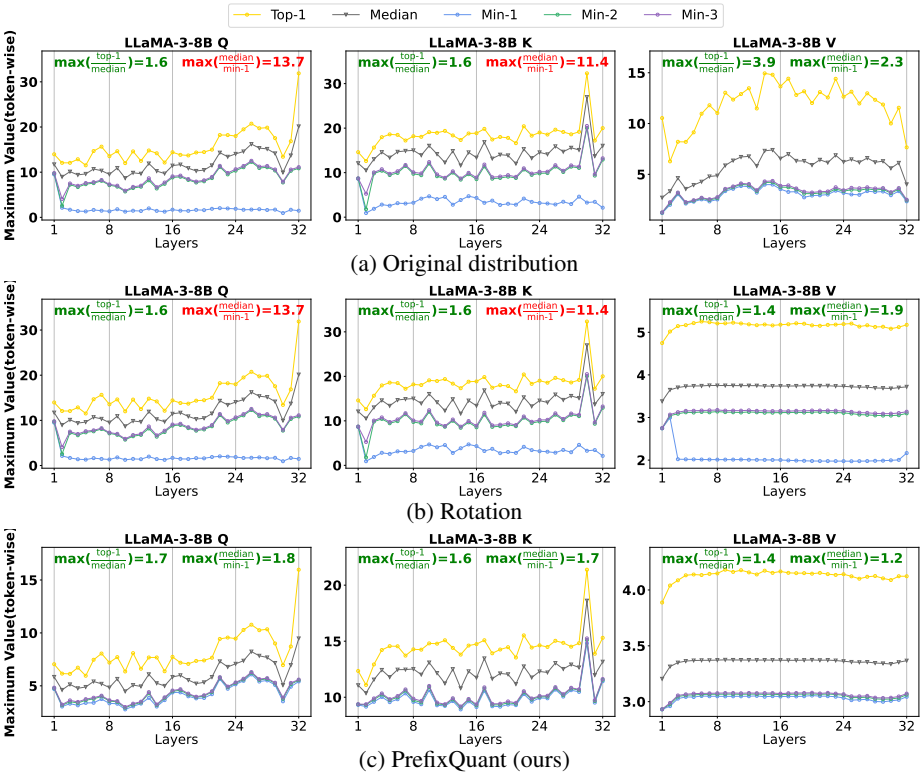


Figure 14: **Distribution of token-wise maximum values for Q/K/V in Llama-3-8B.**

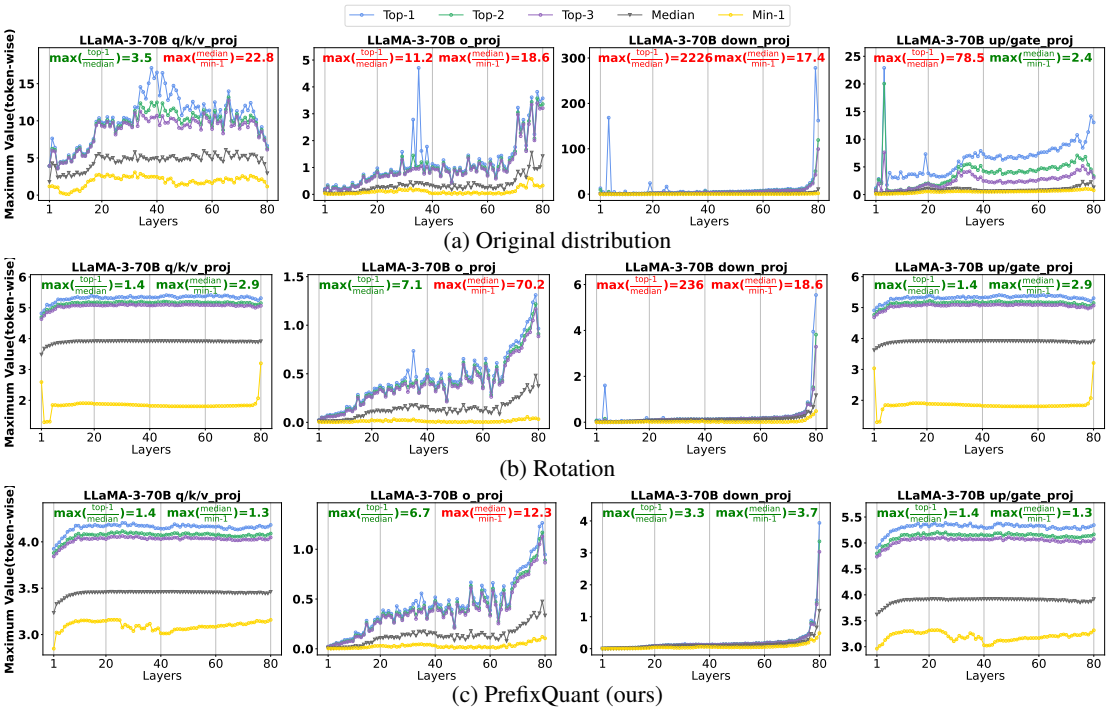


Figure 15: **Distribution of token-wise maximum values for linear layers inputs in Llama-3-70B.**

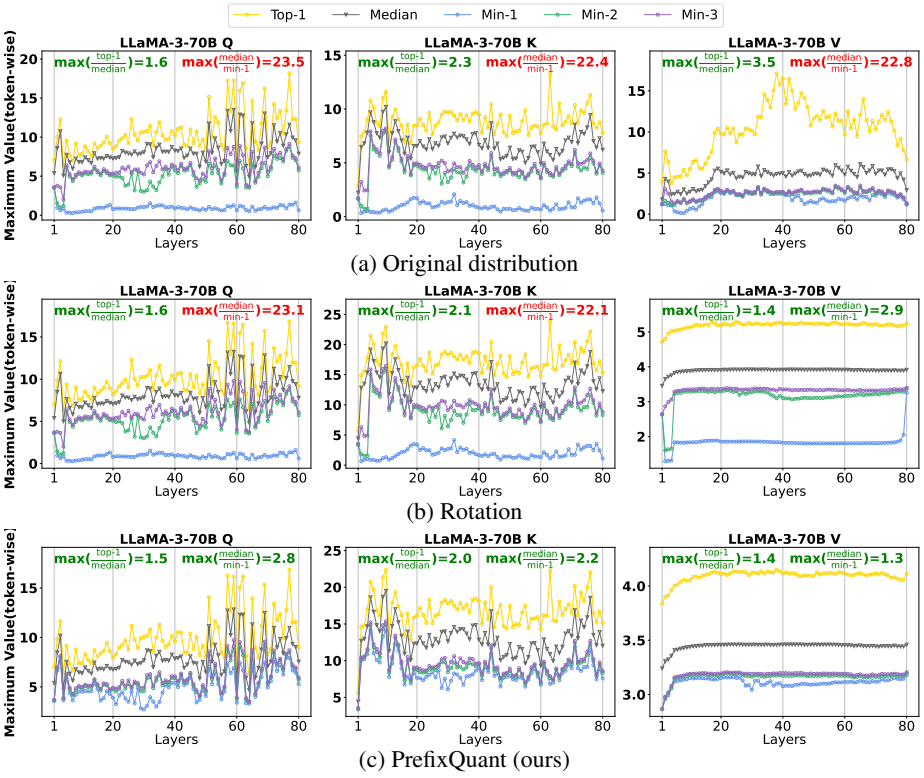


Figure 16: **Distribution of token-wise maximum values for Q/K/V in Llama-3-70B.**

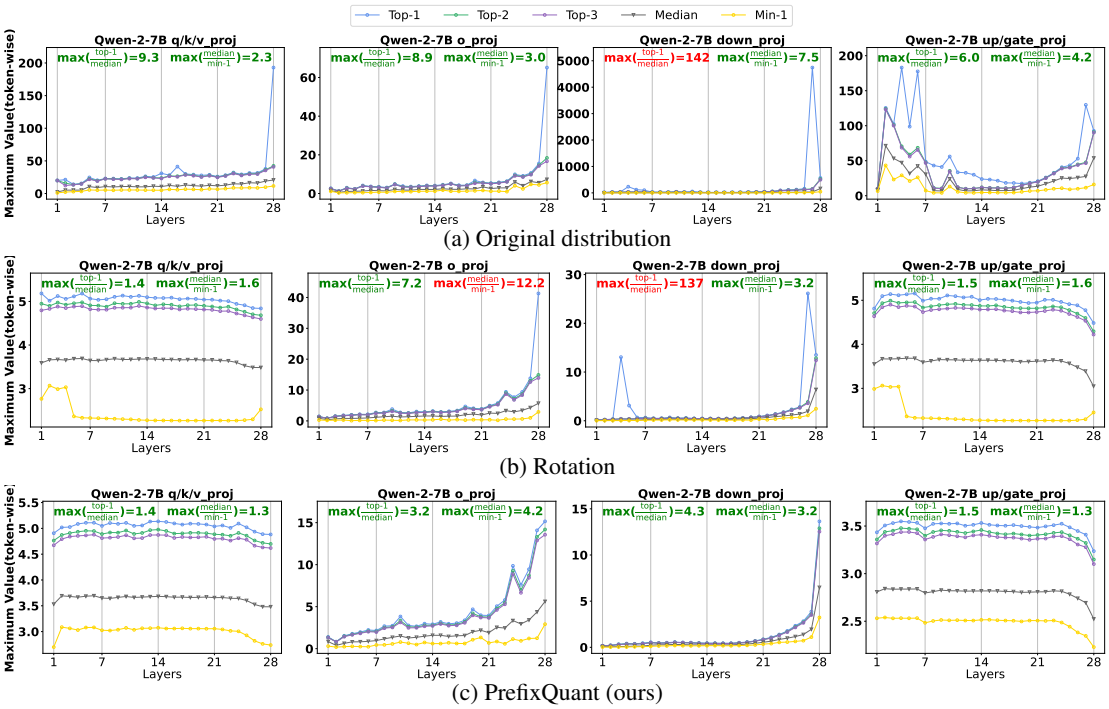


Figure 17: **Distribution of token-wise maximum values for linear layers inputs in Qwen-2-7B.**

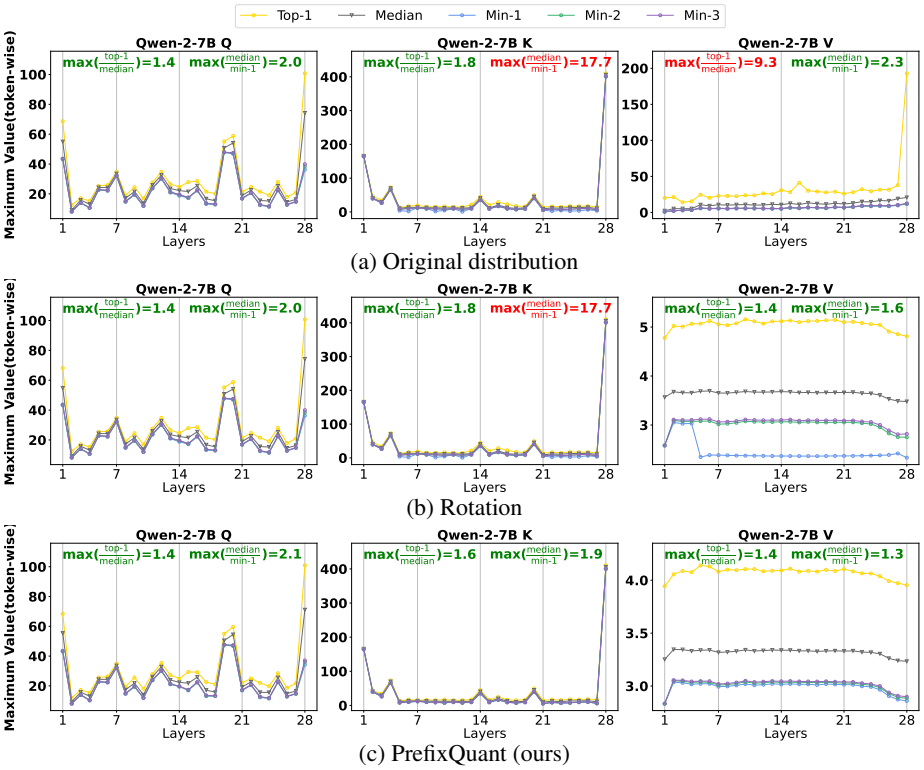


Figure 18: **Distribution of token-wise maximum values for Q/K/V in Qwen-2-7B.**

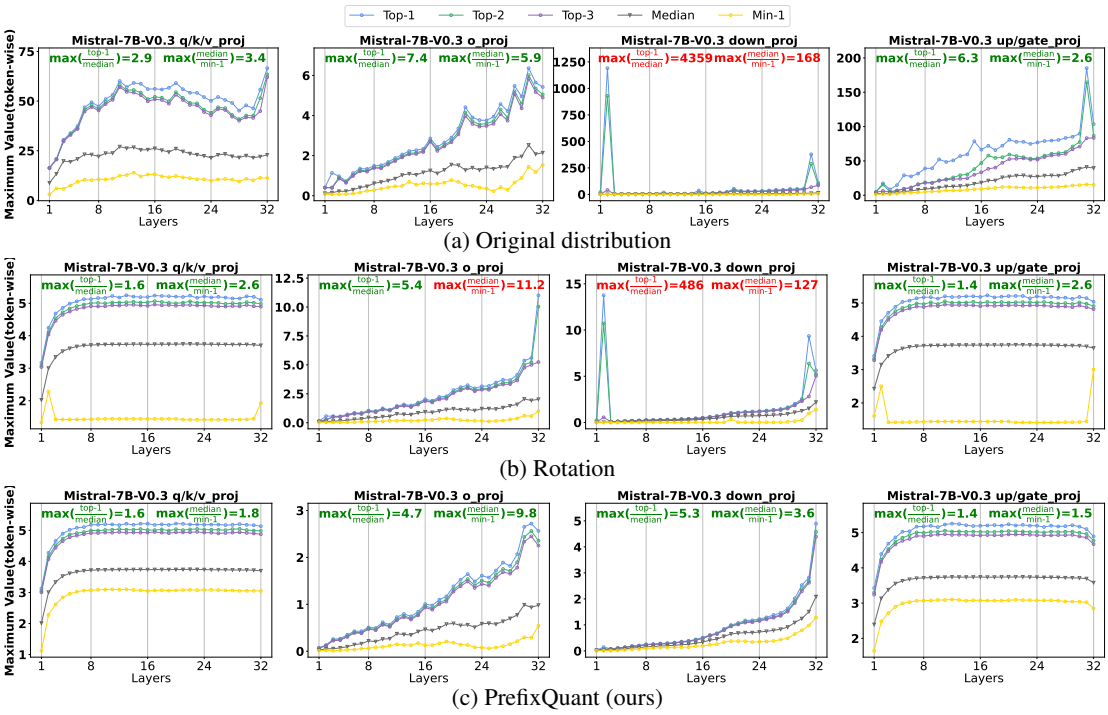


Figure 19: **Distribution of token-wise maximum values for linear layers inputs in Mistral-7B-v0.3.**

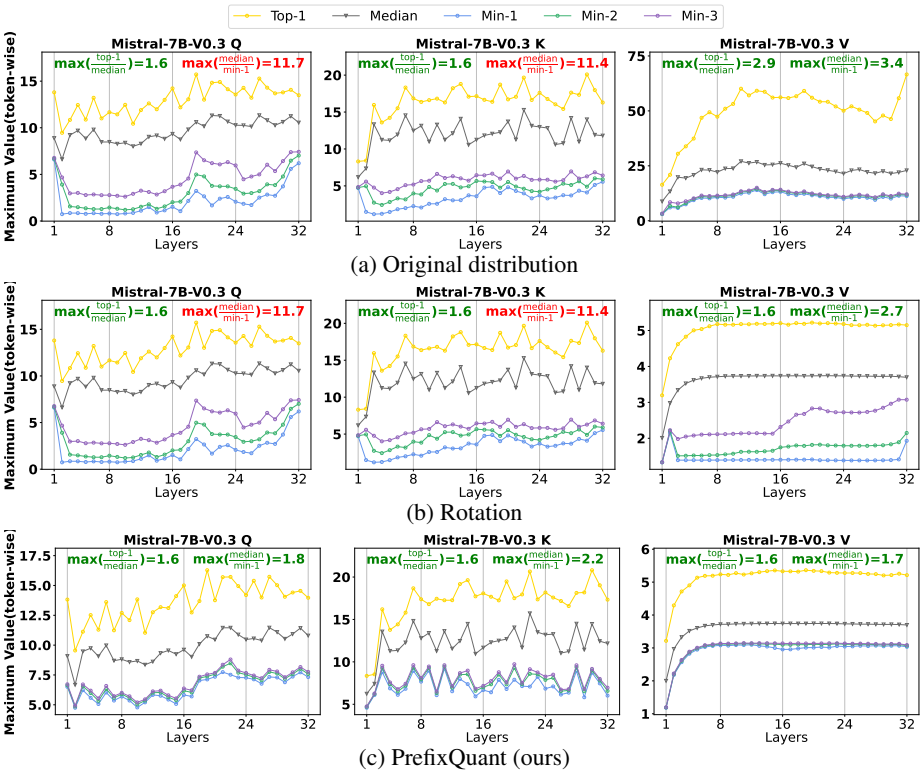


Figure 20: **Distribution of token-wise maximum values for Q/K/V in Mistral-7b-v0.3.**